



01211373 Machine Learning and Programming for Industry

DEEP LEARNING

PyTorch Mechanics

Lecture 8 — A Regression Project on Thermal Drift

Last Week → This Week

WEEK 7 RECAP

- Tensors track gradients; autograd does the calculus
- `nn.Module` defines the model
- forward → loss → backward → step
- On tabular data, the MLP merely tied the random forest

TODAY

- A regression problem where the network earns its keep
- Choosing a loss function — and why it can matter more than architecture
- Optimizers and learning rate
- Validation monitoring and early stopping
- Saving a trained model for deployment

Today's Problem: Spindle Thermal Drift



A machining centre's spindle heats up as it runs. Bearings, motor and column all expand — at different rates — and the tool tip drifts away from where the controller thinks it is.

The Symptom

Parts machined after a cold start come out to a different size than parts machined three hours in — on the same program.

The Scale

Tip drift of 40–80 μm over a warm-up period. A precision job may have a $\pm 5 \mu\text{m}$ tolerance.

The Fix

Predict the drift from temperature sensors, and have the controller apply an equal-and-opposite offset in real time.

This is thermal error compensation — a standard feature on high-end machine tools, and a genuine regression problem.

Framing It as a Learning Problem

Everything on the left is cheap to measure continuously. The thing on the right is expensive to measure in production — which is exactly when a model is worth building.

INPUTS (7 features)

- Front bearing temperature
- Rear bearing temperature
- Motor winding temperature
- Column casting temperature
- Ambient temperature
- Spindle speed (rpm)
- Elapsed runtime (min)



TARGET

Tool-tip displacement

measured in micrometres

A continuous value → regression.

Report error in μm , not R^2 alone — μm is what the machinist and the tolerance spec speak in.

nn.Module: More Than a Container



Subclassing gives you three things Sequential cannot: named sub-blocks you can reuse, logic inside the forward pass, and a model that documents its own structure.

Reusable blocks

Define a small block once (linear + activation + dropout) and stack several. Change it in one place, not five.

Logic in forward()

Branch, skip, concatenate, or reshape mid-network. Week 9's CNNs and Week 10's LSTMs all need this.

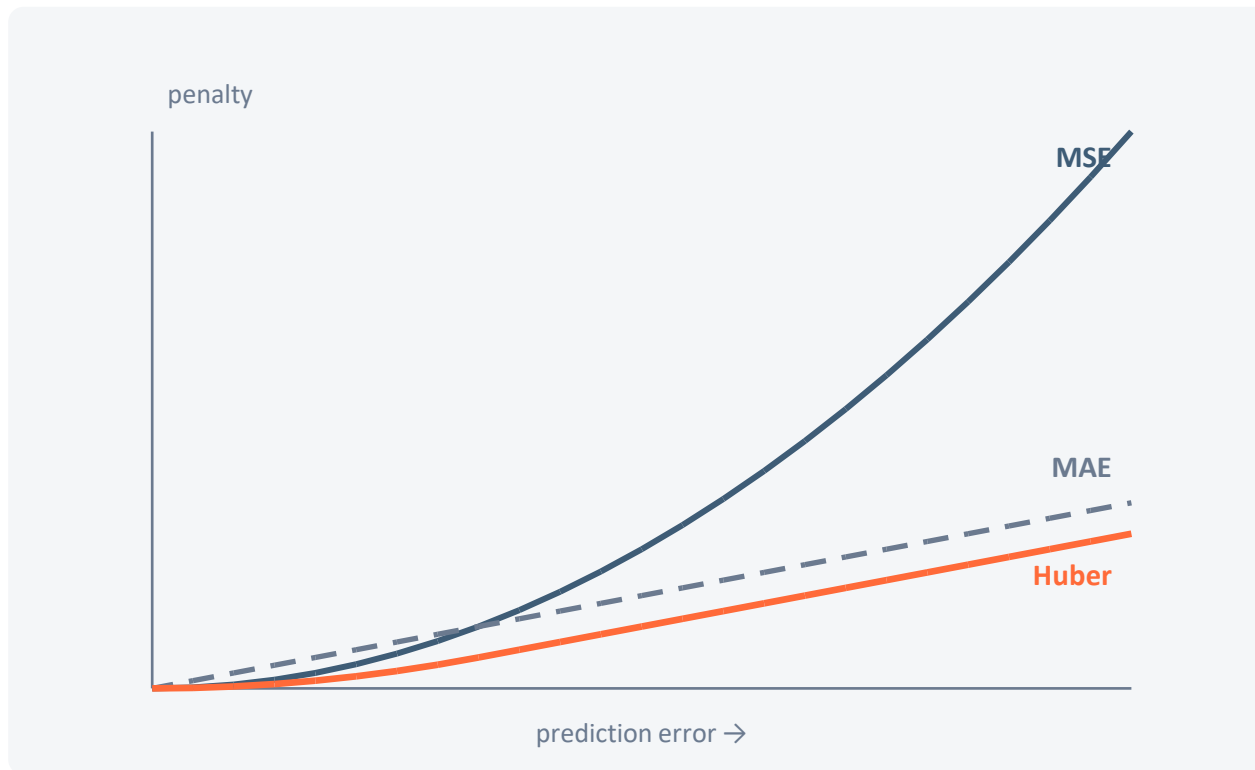
Self-describing

Printing the model prints its architecture; `.parameters()` hands the optimizer everything trainable, automatically.

Choosing a Regression Loss



The loss defines what “wrong” means. On real sensor data that choice is not cosmetic — it decides how much a handful of bad readings can distort the whole model.



MSE

Squares the error — one reading that is 20 μm out counts as much as 400 that are 1 μm out.

MAE

Penalizes linearly. Robust to outliers, but its gradient does not shrink near the optimum.

Huber

Quadratic for small errors, linear for large ones. Smooth training plus outlier resistance.

Why This Is Not an Academic Choice



The displacement probe occasionally glitches — a chip, a splash of coolant, an EMI spike. Roughly 1.5% of readings are badly wrong. Same network, same data, same everything except the loss:

Trained with MSE

0.61 μm

Model chases the glitches

Trained with Huber

0.47 μm

Glitches are down-weighted

RMSE against true displacement on clean test readings, with early stopping active in both runs

A 23% reduction in error — from changing one line. Early stopping already limits how far MSE can chase the glitches; the loss function closes the rest of the gap. Uncompensated drift is 31 μm RMS — both models are inside tolerance, but only one has margin to spare.

Optimizers & Learning Rate



The optimizer decides how to turn a gradient into a weight update. Adam is the sensible default; the learning rate is the one hyperparameter you will always have to think about.

SGD

Takes a fixed-size step along the gradient. Simple and well understood; often needs momentum and careful tuning to converge quickly.

Adam

Adapts a per-parameter step size from the recent history of gradients. Converges quickly with little tuning — use it unless you have a reason not to.

Too low

Loss falls, but glacially — you run out of patience before it converges.

About right

Steady decline, then a clean plateau.

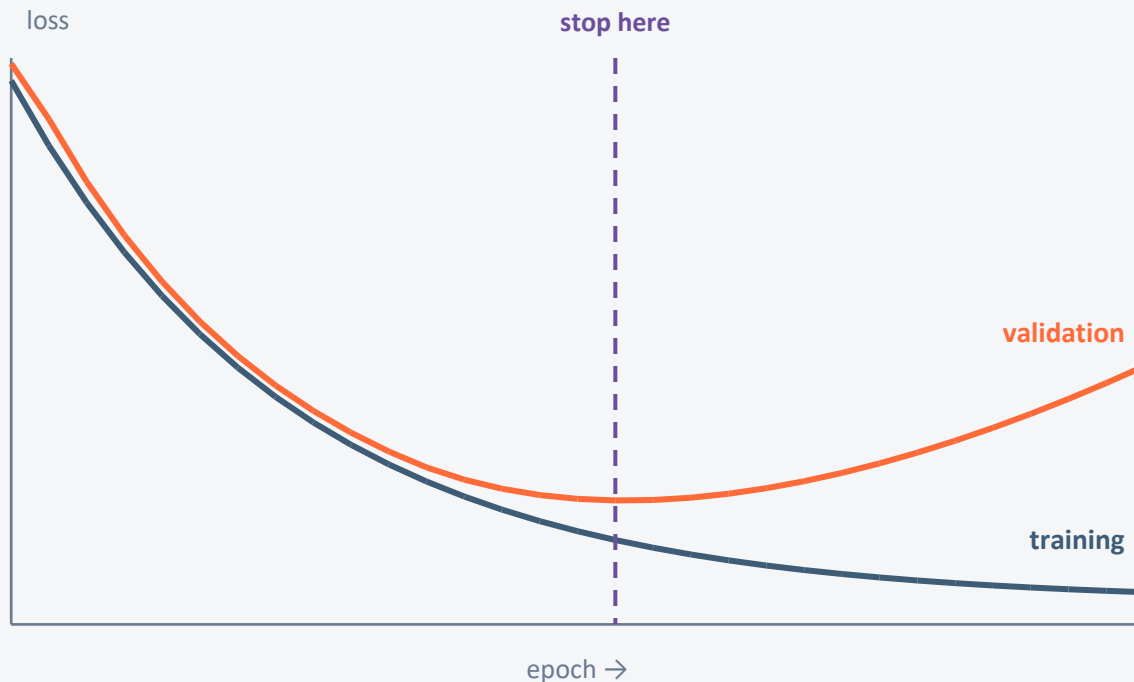
Too high

Loss oscillates or diverges — each step overshoots the minimum.

Watching Validation, and Stopping in Time



Training loss alone cannot tell you when to stop — it falls forever. Hold out a validation split, track it every epoch, and stop when it turns.



Early stopping keeps a copy of the weights from the best validation epoch, and restores them at the end.

“Patience” is how many epochs of no improvement you tolerate before giving up — validation loss is noisy, so a patience of 1 stops far too eagerly.

Three Splits, Not Two

Once validation data starts influencing decisions — when to stop, which architecture — it is no longer an unbiased estimate. Week 6's rule applies unchanged.

~70%

Training

Weights are fitted on this. Seen every epoch.

~15%

Validation

Monitored every epoch to decide when to stop. Never trained on — but it does influence the model.

~15%

Test

Untouched until the very end. Evaluated exactly once.

If you stop training at the best validation epoch, your validation score is now an optimistic estimate — you selected it for being the best. Only the test set gives you an honest number.

Saving a Model for Deployment



A trained model that exists only in a notebook session is not an engineering deliverable. What you ship is the learned parameters — plus everything needed to reproduce the inputs.

The weights

The state dictionary — every learned parameter. This is what training produced.

The architecture

The class definition. Weights are meaningless without the structure they belong to.

The scaler

The training-set means and standard deviations. Feed raw °C to a model trained on scaled inputs and the predictions are nonsense.

Forgetting the scaler is the single most common way a model that worked in the lab fails on the machine.

The Week 8 Workflow, End to End



Notice how much of this is Weeks 2–6 unchanged. Scale on train only, hold out a test set, touch it once. Switching frameworks did not change a single one of those rules — only who writes the loop.

Regression-Specific Pitfalls

Shape mismatch on the target

A target of shape (N,) against predictions of shape (N,1) silently broadcasts into an N×N loss — no error, nonsense training.

Unscaled target

If the target spans 0–80 μm , initial losses are enormous and gradients can blow up. Scaling the target is legitimate — just remember to invert it.

Reporting R^2 only

$R^2 = 0.99$ sounds excellent. Ask what it is in μm , then compare that against the tolerance you must hold.

Validation used twice

Choosing the stopping epoch AND reporting that epoch's score. The second number is not honest.

Scaler left behind

Model saved, scaler forgotten. Predictions on the machine are confidently, consistently wrong.

Today's Lab: Build a Thermal Compensation Model

1

Simulate & explore the machine

Warm-up curves, sensor lags, and the probe glitches.

2

Build a custom regression module

A reusable block, stacked — not a bare Sequential.

3

MSE vs Huber, head to head

Same network, same data. Watch the error halve.

4

Early stopping, then save & reload

Verify the reloaded model predicts identically.

Key Takeaways

- ✓ The loss function encodes what you consider a bad prediction — choose it deliberately.
- ✓ Validation tells you when to stop; only the test set tells you how well you did.
- ✓ A deployable model is weights + architecture + the scaler that made its inputs.

NEXT WEEK

Convolutional Neural Networks

Raw images instead of hand-made features — automated visual inspection on the production line.